CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA DE MINAS GERAIS
ENGENHARIA ELÉTRICA

CHRISTOPHER CÉSAR DE MELO ANDRADE
IAN CAMARGOS LAMOUNIER DUARTE

LABORATÓRIO DE SISTEMAS MICROPROCESSADOS
RELATÓRIO FINAL

Belo Horizonte, 20 de Dezembro de 2025

1. Introdução

A automação de portões em locais movimentados como, por exemplo, em portarias de condomínio, shoppings ou estacionamento em geral é um exemplo clássico da aplicação de sistemas embarcados com a finalidade de garantir segurança, confiabilidade e conforto. Sendo esta a motivação para o desenvolvimento de um sistema de controle para um portão automatizado utilizando um microcontrolador PIC18F4550.

O projeto explora conceitos essenciais de sistemas embarcados, como configuração de registradores, uso de interrupções externas, leitura de entradas digitais e utilização de módulos internos do microcontrolador, incluindo temporizadores (Timer) e o módulo CCP para geração de sinais sonoros. O principal objetivo do trabalho é aplicar, de forma prática, os conhecimentos adquiridos na disciplina, demonstrando o funcionamento integrado do hardware e do software, bem como a importância do tratamento adequado de eventos críticos, como situações de emergência e detecção de presença, garantindo segurança e confiabilidade ao sistema.

Tabela 1: Visão geral do microcontrolador e das configurações utilizadas.

Componente	Port/Pin do Microcontrolador	Função
Botão de comando de usuário	RB1 (External Interrupt INT1)	Inicia ou para o movimento do portão.
Botão do sensor de presença	RB0 (External Interrupt INT0)	Simula um sensor de presença para parar o portão por segurança.
Botão de parada de emergência	RB2 (External Interrupt INT2)	Ativa uma parada imediata do portão e ativa os alarmes.
Display da posição do portão	PORTD (8 LEDs)	Representa visualmente a posição do portão durante a operação
Indicador de estado (presença)	LATCbits.LATC1	Indica que o sensor de presença está ativo.
Indicador de emergência	LATEbits.LATE0	Providencia um sinal de emergência visual.
Indicador de estado normal do sistema	LATAbits.LATA5	Indica o estado normal do sistema, muda durante uma emergência.
Alarme	RC2 (saída do módulo CCP1)	Gera tons distintos para operação normal e alertas de emergência.

2. Metodologia

O sistema foi desenvolvido com base em uma lógica de controle estruturada como uma máquina de estados, responsável por definir as condições de abertura, fechamento, parada e retomada do movimento do portão. As entradas digitais foram configuradas para receber sinais provenientes de 3 (três) chaves mecânicas sendo elas: o botão do controle do portão, onde cada usuário possui o seu para abrir, parar ou fechar o portão; A chave do sensor de presença, a qual define se há ou não, presença nas proximidades do portão e da botoeira de emergência, a qual trava todo o sistema, por questões de segurança. Todas as chaves foram tratadas por interrupção externa: INT0, INT1 e INT2. A ordem de prioridade, apesar de que não foi definida prioridade por software, é, do maior para menor prioridade, Botoeira de emergência, sensor de presença e botão dos usuários. Dessa forma, a segurança é garantida, permitindo que o usuário não acione o portão em casos de pessoas próximas ou emergência.

O movimento do portão foi simulado por meio de um padrão de deslocamento de LEDs conectados à PORTA D, representando visualmente a abertura e o fechamento. Durante o movimento, um sinal sonoros é gerado por um buzzer piezo elétrico ativo, utilizando o módulo CCP (Capture/Compare/PWM) do microcontrolador, em conjunto do TIMER2, com o fim de simular um alerta que existe em sistemas de portão automático reais.

O tratamento de interrupções externas foi empregado com implementação de debounce por software, devido ao uso de chave mecânica. Todos os registradores necessários para o funcionamento dos módulos de I/O, interrupções, temporizadores e CCP foram configurados conforme especificado no datasheet do microcontrolador. Essas configurações podem ser observadas na Tabela 2.

Tabela 2: Descrição de todos os registradores utilizados e como foram configurados

Tabela dos registradores		
Registrador	**Configuração**	
INTCON	0xF0 (11110000)	Habilita Global Interrupts (GIE), Peripheral Interrupts (PEIE), o Timer0 Overflow Interrupt (TMR0IE), o External Interrupt 0 (INT0IE), e o PORTB Change Interrupt (RBIE)
INTCON2	0x70 (01110000)	Seta os bits INTEDG0, INTEDG1, e INTEDG2 a 1, configurando as três interrupções externas para ativar borda de subida. Seta o bit RBPU em 0, para ativar os resistores de pull-up no PORTB.
INTCON3	0x18 (00011000)	Habilita as fontes External Interrupt 1 (INT1IE) e External Interrupt 2 (INT2IE).
T0CON	0x06 (00000110)	Configura o timer para gerar o interrupt periódico que ativa o indicador LED de emergência, quando este estiver ativo. Opera em modo 16-bits, usa o ciclo de clock interno, e ativa um prescaler 1:128.
TRISB	0xFF	Configura toda a pinagem PORTB como entrada. Será utilizada para ler os push buttons RB0, RB1 e RB2.
TRISC	0x00	Configura toda a pinagem PORTC como saída, para os LEDs bicolores e a saída do CCP1 para o buzzer.
TRISD	0x00	Configura toda a pinagem PORTD como saída. Estes serão utilizados para os LEDs de visualização de posição.

TRISE	RE0 = 0 e RA5 = 0	Configura os pinos RE0 e RE5 como saída para os indicadores de emergência e energização,

Módulos periféricos utilizados:

1. **Módulo de interrupção externa (INT0, INT1, INT2)**: uma borda de subida nos pinos dos botões RB0 (presença), RB1 (usuário) ou RB2 (emergência) ativa a rotina de interrupção *isr*, cujo único dever é setar uma flag dependendo da interrupção, através de uma máquina de estados. A lógica dos botões é então executada sequencialmente na função *main*.
2. **Módulo Timer0**: é configurado para gerar uma interrupção periódica. Quando a emergência está ativa, a função *isr* executa essa interrupção ao trocar o estado do LED de emergência. Isso mantém o LED em estado consistente, sem piscar, permitindo que o loop principal execute outras tarefas sem ser parado por funções de delay.
3. **Módulo CCP e Timer2**: é utilizado para produzir alarmes sonoros no buzzer via PWM. O Timer2 é configurado para dar o tempo base que determina o período do sinal PWM. A função *tune* gera este processo ao manipular os registradores PR2 (período) e CCPR1L/CCP1CON (duty cycle). A função *calc_PR2* calcula o valor necessário de PR2 para uma dada frequência, gerando uma onda quadrada no pino de saída do CCP1 para ativar o buzzer, através da implementação da fórmula do datasheet:

$$\text{PWM Period} = [(PR2) + 1] \cdot 4 \cdot T_{OSC} \cdot (\text{TMR2 Prescale Value})$$

3. Resultados

A metodologia adotada permitiu integrar hardware e software de maneira organizada, possibilitando a simulação de um sistema real de automação de portão, atingindo resultados satisfatórios.

Todas as funcionalidades previstas no projeto, bem como os mecanismos de segurança e bloqueio estabelecidos pelas regras de controle foram atendidas. Durante os testes, foi possível verificar o correto funcionamento do acionamento pelo usuário, da detecção de presença e do botão de emergência, observando-se que o comportamento do portão seguiu rigorosamente a lógica e prioridade definida no software.

Quando o sensor de presença esteve ativo, o sistema impediu qualquer movimentação do portão, mesmo diante de tentativas de acionamento pelo usuário, garantindo que a abertura ou o fechamento somente ocorresse após a desativação do sensor. De forma análoga, ao acionar o botão de emergência, todo o funcionamento do sistema foi imediatamente interrompido, sendo executado o alarme sonoro de emergência, permanecendo o sistema bloqueado enquanto a condição de emergência estivesse presente.

Os testes também confirmaram que, ao cessar uma condição de bloqueio causada pelo sensor de presença ou pela emergência, o portão retomou automaticamente o movimento no mesmo sentido em que se encontrava antes da interrupção, conforme especificado no projeto. Além disso, durante o movimento do portão, o padrão visual dos LEDs e a geração de sinais sonoros ocorreram de maneira sincronizada, indicando corretamente o estado de operação do sistema.

Dessa forma, os resultados obtidos demonstram que todas as regras de funcionamento e segurança propostas foram corretamente implementadas e respeitadas, validando a lógica de controle adotada e o correto uso dos recursos do microcontrolador.

4. Conclusão

A partir do desenvolvimento e da implementação do sistema de portão automatizado utilizando um PIC18F4550, foi possível aplicar de forma prática os conceitos fundamentais abordados na disciplina, como configuração de registradores, uso de portas de entrada e saída, tratamento de interrupções e utilização de módulos internos do microcontrolador. O projeto demonstrou que a organização do software em estados bem definidos é essencial para garantir um comportamento previsível, seguro e confiável em sistemas embarcados.

5. Anexo

Código completo:

```
#include <xc.h>

#pragma config CPUDIV       = OSC1_PLL2
#pragma config FOSC         = HS
#pragma config WDT          = OFF
#pragma config PBADEN       = OFF
#pragma config LVP          = OFF
#pragma config MCLRE        = ON
#pragma config PWRT         = ON
#pragma config XINST        = OFF

#define _XTAL_FREQ 20000000UL

#define T2_PRESCALE_1   1
#define T2_PRESCALE_4   4
#define T2_PRESCALE_16  16

#define ON              (1)
#define OFF             (0)

#define GATE_CLOSED    (1)
#define GATE_IN_MIDDLE (0)
#define GATE_OPENED    (-1)

#define GATE_STOPED    (0)
#define GATE_CLOSING   (1)
#define GATE_OPENING   (2)

#define REASON_LIMIT_SWITCH (0)
#define REASON_PRESENCE     (1)
#define REASON_EMERGENCY    (2)
#define REASON_BY_USER      (3)

static unsigned char calc_PR2(float freq, unsigned char *ps_val, unsigned char *t2con_bits);
void tune(unsigned char channel, float freq);
void noTone(void);
void delay_s(unsigned int seconds);
void delay_ms(unsigned int milliseconds);
void Play_Alarm_Gate();
void Play_Alarm_Emergency();
void handle_INT0(void);
```

```c
void handle_INT1(void);
void handle_INT2(void);

int     rise_fall_time;
int     steps;
int     f_max;
int     f_min;
float   delay_time;
float   step_size;

unsigned char  gate_pattern        = 0xFF;
signed char    gate_static_state   = GATE_CLOSED;
signed char    gate_dinamic_state  = GATE_STOPED;
signed char    last_dinamic_state  = GATE_CLOSING;
unsigned char  stop_reason         = REASON_LIMIT_SWITCH;

unsigned char flag_INT0 = 0;
unsigned char flag_INT1 = 0;
unsigned char flag_INT2 = 0;

unsigned char alarm_gate_enable = 0;
unsigned char is_presence       = 0;
unsigned char is_emergency      = 0;


void interrupt isr()
{
    if(INTCONbits.TMR0IF)
    {
        INTCONbits.TMR0IF = OFF;
        TMR0  = 65536 - (_XTAL_FREQ / 512);

        LATEbits.LATE0 = !LATEbits.LATE0;
    }

    if(INTCONbits.INT0IF)
    {
        INTCONbits.INT0IF = OFF;
        flag_INT0 = 1;
    }

    if(INTCON3bits.INT1IF)
    {
        INTCON3bits.INT1IF = OFF;
        flag_INT1 = 1;
    }

    if(INTCON3bits.INT2IF)
    {
        INTCON3bits.INT2IF = OFF;
        flag_INT2 = 1;
    }
}

void main(void)
{
    INTCON  = 0xF0;
    INTCON2 = 0x70;
    INTCON3 = 0x18;

    T0CON  = 0x06;
    TMR0  = 65536 - (_XTAL_FREQ / 512) ;

    TRISD = 0x00;
    TRISB = 0xFF;
    TRISC = 0x00;

    TRISEbits.RE0 = 0;
    TRISAbits.RA5 = 0;
    LATAbits.LATA5 = 1;

    LATD  = ~gate_pattern;

    while(1)
    {
        if(flag_INT2)
        {
```

```
            __delay_ms(5);
            handle_INT2();
            flag_INT2 = 0;
        }
        Play_Alarm_Emergency();
        Play_Alarm_Gate();
        if(flag_INT0)
        {
            __delay_ms(5);
            handle_INT0();
            flag_INT0 = 0;
        }

        if(flag_INT1)
        {
            __delay_ms(5);
            handle_INT1();
            flag_INT1 = 0;
        }

        if(gate_dinamic_state == GATE_CLOSING)
        {
            if(gate_pattern != 0xFF)
            {
                gate_pattern = (gate_pattern >> 1) | 0x80;
                alarm_gate_enable = ~alarm_gate_enable;
                LATD = ~gate_pattern;
                for (unsigned int i = 0; i < 750; i++)
                {
                    if (gate_dinamic_state == GATE_STOPED) break;
                    __delay_ms(1);
                }
            }
           else
            {
                gate_static_state  = GATE_CLOSED;
                last_dinamic_state = GATE_CLOSING;
                gate_dinamic_state = GATE_STOPED;
                alarm_gate_enable  = 0;
                stop_reason        = REASON_LIMIT_SWITCH;
            }
        }

        else if(gate_dinamic_state == GATE_OPENING)
        {
            if(gate_pattern != 0x80)
            {
                gate_pattern <<= 1;
                LATD = ~gate_pattern;
                alarm_gate_enable = ~alarm_gate_enable;
                for (unsigned int i = 0; i < 750; i++)
                {
                    if (gate_dinamic_state == GATE_STOPED) break;
                    __delay_ms(1);
                }
            }
            else
            {
                gate_static_state  = GATE_OPENED;
                last_dinamic_state = GATE_OPENING;
                gate_dinamic_state = GATE_STOPED;
                alarm_gate_enable  = 0;
                stop_reason        = REASON_LIMIT_SWITCH;
            }
        }

        else
        {
            gate_static_state = GATE_IN_MIDDLE;
        }
    }

    return;
}

/* Se o sensor de presença ativar */
void handle_INT0(void)
```

```
{
    if(is_emergency)
        return;

    if(is_presence)
    {
       is_presence = 0;
       LATCbits.LATC1 = 0;
    }




    else
    {
        is_presence = 1;
        LATCbits.LATC1 = 1;

        if(gate_dinamic_state != 0)
        {
           last_dinamic_state = (gate_dinamic_state == GATE_OPENING) ? GATE_CLOSING : GATE_OPENING;
        }

        stop_reason        = REASON_PRESENCE;
        gate_dinamic_state = GATE_STOPED;
        alarm_gate_enable = 0;
    }
}

/* Botão apertado pelo usuário */
void handle_INT1(void)
{
    if(is_emergency || is_presence)
        return;

    LATCbits.LATC6 = !LATCbits.LATC6;

    if(gate_dinamic_state == GATE_CLOSING || gate_dinamic_state == GATE_OPENING)
    {
        alarm_gate_enable  = 0;
        last_dinamic_state = gate_dinamic_state;
        gate_dinamic_state = GATE_STOPED;
        stop_reason = REASON_BY_USER;
    }

    else if(gate_dinamic_state == GATE_STOPED)
    {
        if(last_dinamic_state == GATE_CLOSING)
        {
            gate_dinamic_state = GATE_OPENING;
        }

        else if(last_dinamic_state == GATE_OPENING)
        {
            gate_dinamic_state = GATE_CLOSING;
        }
        last_dinamic_state = GATE_STOPED;
    }
}

/* Se o botão de emergência for acionado */
void handle_INT2(void)
{
    if(is_emergency)
    {
       is_emergency = 0;
       LATEbits.LATE0 = 0;
       LATAbits.LATA5 = 1;
       T0CONbits.TMR0ON = 0;
    }

    else
    {
        is_emergency = 1;
        LATEbits.LATE0 = 1;
        LATAbits.LATA5 = 0;
        T0CONbits.TMR0ON = 1;
```

```c
        if(gate_dinamic_state != 0)
        {
           last_dinamic_state = (gate_dinamic_state == GATE_OPENING) ? GATE_CLOSING : GATE_OPENING;
        }

        stop_reason        = REASON_EMERGENCY;
        gate_dinamic_state = GATE_STOPED;
        alarm_gate_enable = 0;
    }
}


static unsigned char calc_PR2(float freq, unsigned char *ps_val, unsigned char *t2con_bits)
{
    unsigned long pr2;

    *ps_val = T2_PRESCALE_1;
    *t2con_bits = 0b00000000;
    pr2 = (_XTAL_FREQ / (4UL * (unsigned int) freq * (*ps_val))) - 1UL;
    if (pr2 <= 255UL) return (unsigned char)pr2;

    *ps_val = T2_PRESCALE_4;
    *t2con_bits = 0b00000001;
    pr2 = (_XTAL_FREQ / (4UL * (unsigned int) freq * (*ps_val))) - 1UL;
    if (pr2 <= 255UL) return (unsigned char)pr2;

    *ps_val = T2_PRESCALE_16;
    *t2con_bits = 0b00000010;
    pr2 = (_XTAL_FREQ / (4UL * (unsigned int) freq * (*ps_val))) - 1UL;
    if (pr2 <= 255UL) return (unsigned char)pr2;

    return 255;
}

void tune(unsigned char channel, float freq)
{
    unsigned char ps_val, t2bits;
    unsigned char pr2;
    unsigned int duty_10bit;

    pr2 = calc_PR2(freq, &ps_val, &t2bits);

    PR2 = pr2;

    duty_10bit = (unsigned int)(pr2 + 1) * 2;

    if (channel == 1)
    {
        TRISCbits.TRISC2 = 0;
        CCPR1L = (unsigned char)(duty_10bit >> 2);
        CCP1CONbits.DC1B = (unsigned char)(duty_10bit & 0x03);
        CCP1CONbits.CCP1M = 0b1100;
    }

    else if (channel == 2)
    {
        TRISCbits.TRISC1 = 0;
        CCPR2L = (unsigned char)(duty_10bit >> 2);
        CCP2CONbits.DC2B = (unsigned char)(duty_10bit & 0x03);
        CCP2CONbits.CCP2M = 0b1100;
    }
    else
        return;

    TMR2 = 0;
    T2CON = (T2CON & 0b11111000) | (t2bits & 0x03);
    T2CONbits.TMR2ON = 1;
}

void noTone(void)
{
    CCP2CON = 0x00;             // Desabilitar CCP2
    T2CONbits.TMR2ON = 0;       // Parar Timer2
    CCPR2L = 0;                 // Duty cycle 0
}
```

```
void delay_ms(unsigned int milliseconds)
{
    while(milliseconds > 0)
    {
        __delay_ms(1);
        milliseconds--;
    }
}




void Play_Alarm_Gate()
{
    if(alarm_gate_enable)
        tune(1, 1200);
    else
        noTone();
}

void Play_Alarm_Emergency()
{
    if(!is_emergency)
        return;

    for(int i = 0; i < 10; i++)
    {
        if(!is_emergency)
            break;

        for(long float f = 3000; f > 300; f = f * 0.85)
        {
            if(!is_emergency)
                break;
            tune(1, f);
            __delay_ms(15);
        }
    }

    if(!is_emergency)
        return;

    noTone();
    __delay_ms(10);

    for(int i = 0; i < 10; i++)
    {
        if(!is_emergency)
           break;
        for (float f = 3000; f > 100; f = f * 0.85)
        {
            if(!is_emergency)
                break;

            tune(1, 2 * f);
            __delay_ms(5);
            tune(1, f);
            __delay_ms(5);
        }
    }

    if(!is_emergency)
        return;

    noTone();
    __delay_ms(10);

    rise_fall_time = 90;
    steps = 10;
    f_max = 1000;
    f_min = 500;
    delay_time = rise_fall_time/steps;
    step_size = 0.93;

    for(int i = 0; i < 20; i++)
    {
```

```
        if(!is_emergency)
           break;
        for (float f = f_max;f > f_min; f *= step_size)
        {
            if(!is_emergency)
                break;

            tune(1, f);
            delay_ms((unsigned int) delay_time);
        }
    }


    if(!is_emergency)
        return;

    noTone();
    __delay_ms(10);

    rise_fall_time = 1000;
    steps = 100;
    f_max = 1500;
    f_min = 500;
    delay_time = rise_fall_time/steps;
    step_size = 1.05;

    for(int i = 0; i < 10; i++)
    {
        if(!is_emergency)
            return;
        for (float f =f_min;f<f_max;f*=step_size)
        {
            if(!is_emergency)
                return;
            tune(1, f);
            delay_ms((unsigned int) delay_time);
        }
    }

    if(!is_emergency)
        return;

    noTone();
    __delay_ms(10);
}
```